Cyber Threat Intelligence for Connected Vehicles: Graph-Based Attack Attribution and Risk Ranking on In-Vehicle CAN and V2X Telemetry

Yuvraj Singh, Likhith Sai Ande, Charan Krishna Y, Nandam Pardhu Kumar, and Kaushik Kaligotla
Dept. of Computer Science and Engineering, SRM University-AP
Amaravati, Andhra Pradesh, India
Corresponding author: satishyuvraj.singh3178@gmail.com

***Abstract*—Intrusion detectors for connected vehicles usually return a class label and nothing more. They do not say which adversary technique an alert corresponds to or which vehicle assets are exposed. This paper examines whether linking detector output to cyber threat intelligence (CTI) improves attack attribution and alert prioritization on two public benchmarks: CICIoV2024 for in-vehicle CAN traffic and VeReMi for vehicle-to-everything (V2X) beacon misbehavior. We build a 28-node vehicle-threat knowledge graph that connects observed attack classes to MITRE ATT&CK techniques, CWE weaknesses, and vehicle assets, and we train a graph-convolutional classifier (CTI-GNN) that attributes each event to an attack class and produces a contextual risk score. Against five baselines and a graph-free ablation, evaluated over 20 seeds with validation-only tuning and a run-level split on VeReMi, CTI-GNN reaches a macro-F1 of 0.306 ± 0.049. This is below random forest (0.460) and XGBoost (0.456), and its gain over a feed-forward network without the graph (0.297) is not statistically significant (paired t-test, $p = 0.372$). On CAN data every model scores close to 1.0, and a leakage audit shows that this ceiling depends on the sampling protocol: under a contiguous time-block split, the macro-F1 of random forest falls to 0.513. Alert rankings are also sensitive to the heuristic weights, with Kendall's τ dropping from 0.88 to 0.34 as the weights are perturbed by 5% to 30%. We conclude that the main value of the graph lies in interpretable attribution and context rather than in detection accuracy, and we report the negative results alongside the positive ones.**

***Index Terms*—connected vehicles, cyber threat intelligence, MITRE ATT&CK, graph neural network, CAN bus, V2X misbehavior detection, intrusion detection**

I. Introduction

Modern vehicles exchange data over two very different channels. Inside the vehicle, electronic control units (ECUs) communicate on a Controller Area Network (CAN) bus whose frames carry no built-in authentication, so a compromised component can address every other component on the same bus [1]. Between vehicles, safety applications rely on periodic basic safety messages (BSMs) in which each sender reports its own position and speed. Both channels can be abused, and a large body of work has proposed detectors for it [2]. Most of that work stops at a class label.

A label is a weak input for a security operations team. An analyst who receives "spoofing attack" still has to decide which technique this corresponds to, which weakness it exploits, which ECUs are affected, and how urgently to respond. Shared vocabularies such as MITRE ATT&CK [3] and the Common Weakness Enumeration (CWE) [4] answer these questions for conventional IT systems, but they are rarely connected to the output of vehicular detectors. It is also not obvious that connecting them helps. Encoding threat knowledge inside the model could improve attribution, or it could add parameters that the available data cannot support.

This paper tests that question directly. We construct a heterogeneous vehicle-threat graph, train a graph-convolutional classifier that consumes it, and compare the result against standard baselines on two public datasets. The evaluation protocol is deliberately conservative: hyperparameters are tuned on validation data only, VeReMi is split by whole simulation run, all results are averaged over 20 seeds, and the CAN results are subjected to leakage checks. The contributions are as follows.

- A vehicle-threat knowledge graph with 28 nodes and 46 edges linking CAN and V2X attack events to ATT&CK techniques, CWE weaknesses, and vehicle assets, together with a mapping of ten attack classes to techniques.
- CTI-GNN, a graph-convolutional model with two heads, one for attack attribution and one for contextual risk scoring.
- A multi-seed comparison with five baselines and an ablation without the graph, including paired significance tests and a per-class analysis.
- An audit of the CAN benchmark for duplicate and order-dependent leakage, and a sensitivity analysis of the heuristic risk ranking.

The results are mixed. CTI-GNN does not beat tree ensembles on V2X data, its advantage over a graph-free network is not statistically reliable, and the CAN benchmark is close to saturated under the usual sampling protocol.

II. Background

A. Datasets

CICIoV2024 [5] contains CAN traffic captured from a real vehicle, with benign frames, a denial-of-service (DoS) attack, and four spoofing attacks that target the gas pedal, engine speed (RPM), vehicle speed, and steering wheel signals. VeReMi [6] is a simulated V2X dataset in which attacker

vehicles broadcast falsified BSMs. Its five attack types are constant position, constant offset, random position, random offset, and eventual stop.

B. Threat taxonomies

ATT&CK describes adversary behavior as tactics and techniques [3]. Techniques with identifiers of the form T0xxx belong to the ICS matrix, while T1xxx identifiers belong to the Enterprise matrix. Our mapping uses techniques from both matrices. CWE identifiers [4] describe the underlying weakness that an attack exploits.

III. Related Work

Machine-learning detectors for VeReMi commonly use plausibility checks as input features. So et al. [7] fed such features to KNN and SVM classifiers and reported that the classifiers improved the precision of the plausibility checks they were built on. For in-vehicle networks, the survey by Wu et al. [2] describes the constraints that CAN-bus detectors must meet, including real-time communication. These works concentrate on detection accuracy and operating constraints. This paper asks whether linking detector output to threat-intelligence context adds measurable value.

Evaluation practice is a second concern. Arp et al. [8] identify ten recurring pitfalls in learning-based security studies, including sampling bias, and show that they are widespread in the literature. Our run-level split on VeReMi and our leakage audit on CICIoV2024 are responses to this concern. Outside security, Grinsztajn et al. [9] found that tree-based models remain state-of-the-art on tabular data of medium size, around 10,000 samples. Our datasets are larger, so we use that result as context rather than as direct support.

IV. Data and Experimental Protocol

A. CAN bus data (CICIoV2024)

The decimal release contains 1,408,219 frames in six files: 1,223,737 benign frames and 74,663, 9,991, 54,900, 24,951, and 19,977 frames for DoS, gas, RPM, speed, and steering-wheel attacks respectively. To allow repeated runs on CPU hardware, we sampled 10,000 frames per class (all 9,991 for the gas class), giving 59,991 frames. These were split into stratified 70/15/15 partitions of 41,993 training, 8,999 validation, and 8,999 test frames. The feature scaler was fit on the training partition only.

Each frame is described by 17 features: the arbitration ID, five payload statistics (sum, mean, standard deviation, zero-byte count, and Shannon entropy over the eight bytes), the eight normalized payload bytes, and three features that depend on frame order (the run length of identical consecutive IDs, the absolute ID difference to the previous frame, and the byte-wise Hamming distance to the previous frame). The order-dependent features matter for the leakage analysis in Section VI-E.

B. V2X data (VeReMi)

We used all 75 simulation archives of traffic density 3. Records were split by whole simulation run so that no vehicle trajectory is shared between partitions: repetitions 0 to 2 (45 archives, 166,800 records) for training, repetition 3 (15 archives, 54,663 records) for validation, and repetition 4 (15 archives, 56,432 records) for testing. Benign records number 140,893, 42,802, and 43,189 in the training, validation, and test splits; the five attack classes together number 25,907, 11,861, and 13,243. Benign messages make up 76.5% of the test split, so a classifier that always predicts "benign" would reach an accuracy of 0.765. This figure is a useful reference point for the accuracy values reported below.

Ten features are computed for each receiver–sender stream: transmission delay, inter-arrival time Δt (floored at 10^{-4} s), claimed speed, displacement between successive reported positions, the speed implied by that displacement, the absolute difference between claimed and implied speed, acceleration, log received signal strength, and two binary plausibility flags that mark an implied speed above 60 m/s or an acceleration above 12 m/s^2.

C. Models and tuning

We compare the proposed model against five baselines and one ablation: k-nearest neighbors (KNN) [10], random forest (RF) [11], a linear support vector machine (SVM) [12], XGBoost [13], a recurrent network with a sequence length of five built on LSTM units [14], and a feed-forward network without graph or CTI inputs, which serves as the ablation (MLP). Every model received the same search budget of four candidate configurations, and the configuration with the best validation macro-F1 was selected. The test split was not used during selection. The selected settings were $k = 5$ for KNN; depth 10 with 100 trees for RF; $C = 0.01$ with two-fold calibration for the linear SVM; depth 6 with learning rate 0.05 for XGBoost; and hidden size 64 with learning rate 0.003 for the LSTM and the ablation. CTI-GNN used a CTI embedding size of 32 and learning rate 0.003. All final results are the mean and standard deviation over 20 seeds (42, 101, 202, 303, 404, 505, 606, 707, 808, 909, 111, 222, 333, 444, 555, 666, 777, 888, 999, 1234), and 95% confidence intervals for macro-F1 (Student t, 19 degrees of freedom) are reported for VeReMi.

Experiments ran on an Intel Core i5 laptop CPU with 16 GB of memory under Windows 11, with no GPU. The software stack was Python 3.13.5, PyTorch 2.9.1 [15], scikit-learn 1.7.1 [16], XGBoost 3.4.1, NetworkX 3.5 [17], and SciPy 1.16.2 [18].

D. Metrics

We report accuracy, macro-averaged F1, and the false positive rate (FPR), defined as the share of benign records flagged as any attack. Attribution accuracy is the share of true attack records assigned to the correct attack class. Latency is the wall-clock time of one batched forward pass divided by the batch size ($N = 10{,}000$), measured on the host CPU and averaged over seeds.

V. Method

A. Vehicle-threat graph

The graph was built in NetworkX and exported to GraphML and JSON. It has 28 nodes of four types: 9 vehicle assets (including the CAN bus, the engine, throttle, transmission, brake, and steering control units, and the V2X on-board unit), 9 communication events (five CAN attack events and four V2X misbehavior events), 6 ATT&CK techniques, and 4 CWE weaknesses. Its 46 directed edges cover 15 relation types; the largest groups are technique-to-event links (9), technique-to-asset targeting (8), technique-to-weakness exploitation (6), and weakness-to-asset links (6). Fig. 1 shows the structure.

Table I lists the mapping from the ten attack classes to techniques and weaknesses. The mapping, the CWE assignments, and the impact and asset criticality weights were assigned by the authors on the basis of threat-modeling judgment. They are not derived from ISO 26262 hazard analysis or CVSS scores and have not been reviewed by external experts.

B. CTI-GNN

Let $x_e \in \mathbb{R}^{d_{\text{in}}}$ be the feature vector of an incoming event and let $C \in \mathbb{R}^{K \times d_c}$ be a learnable embedding matrix for $K = 10$ CTI nodes. The event is projected into the embedding space,

$$h_e^{(0)} = \text{ReLU}\left(W_{\text{proj}} x_e + b_{\text{proj}}\right), \tag{1}$$

and stacked with the CTI embeddings to form $H^{(0)} = [h_e^{(0)}; C] \in \mathbb{R}^{(1+K)\times d_c}$. Two graph convolution layers [19] follow,

$$H^{(1)} = \text{ReLU}\left(\tilde{A} H^{(0)} W_g^{(1)}\right), \tag{2}$$

$$H^{(2)} = \text{ReLU}\left(\tilde{A} H^{(1)} W_g^{(2)}\right), \tag{3}$$

where $\tilde{A} = D^{-1/2} A D^{-1/2}$ is the symmetrically normalized adjacency matrix of the $(1 + K)$-node graph. The event representation $h_e^{(2)} = H^{(2)}[0, :]$ is concatenated with the mean-pooled CTI context $\bar{c} = \frac{1}{K}\sum_{j=1}^{K} H^{(2)}[j, :]$ to give $z = [h_e^{(2)} \,\|\, \bar{c}] \in \mathbb{R}^{2d_c}$. Two heads read from z:

$$\hat{y} = \text{softmax}\left(W_{\text{attr}} z + b_{\text{attr}}\right), \tag{4}$$

$$\rho(e) = 10\,\sigma\left(W_{\text{risk}} z + b_{\text{risk}}\right) \in [0, 10]. \tag{5}$$

Training minimizes the cross-entropy of the attribution head with an ℓ_2 penalty, $\mathcal{L} = \mathcal{L}_{\text{CE}}(\hat{y}, y) + \lambda \|W\|_2^2$ with $\lambda = 10^{-4}$. Fig. 2 summarizes the architecture. The ablation is a feed-forward network without the graph and the CTI embeddings.

C. Alert prioritization and operator console

Alerts are ranked by a heuristic score equal to the product of an asset criticality weight, the impact weight of the predicted attack class (Table I), and the predicted attack probability. The ranking therefore follows from the author-assigned weights by construction and is not a result learned from data. The learned risk head $\rho(e)$ has no ground-truth severity labels to be trained or evaluated against, and we do not claim that it is validated.

A small Java console (Java 27) loads the exported graph and technique mapping and lists the five highest-priority alerts per network domain with the predicted attack, confidence, score, technique, severity, and affected assets. It was not part of the quantitative evaluation.

VI. Results

A. CAN bus

On the balanced CICIoV2024 test split ($N = 8{,}999$), KNN, RF, the ablation, and CTI-GNN reached an accuracy and macro-F1 of 1.0000 in all 20 seeds. XGBoost scored 1.0000 $\pm$ 0.0001, the LSTM 0.9999 $\pm$ 0.0003, and the linear SVM 0.9997 $\pm$ 0.0002. Because CTI-GNN and the ablation score identically in every seed, the paired tests between them are undefined on this dataset. The benchmark therefore cannot discriminate between the models. Section VI-E examines why.

B. V2X misbehavior

Table II gives the multi-seed VeReMi results and Fig. 3 shows macro-F1 with confidence intervals. Random forest is the strongest model on accuracy and macro-F1, with a macro-F1 of 0.4595, followed by XGBoost at 0.4560, which has the highest attribution accuracy (0.2936). CTI-GNN ranks fourth at 0.3056, behind KNN (0.3819) and ahead of the ablation (0.2968), the LSTM (0.2397), and the linear SVM (0.2238). Its confidence interval overlaps that of the ablation but not that of KNN. The gap to the tree ensembles is about 0.15 macro-F1.

Accuracy differs little between models, ranging from 0.7683 to 0.7927, and KNN, the SVM, and the LSTM sit within 0.005 of the always-benign reference of 0.765. Accuracy is therefore a poor guide on this split, and macro-F1 is the more informative measure. CTI-GNN has a lower FPR (3.2%) than RF (5.3%), XGBoost (6.9%), and KNN (6.3%), but its attribution accuracy is also lower (0.1607 against 0.2875 for RF). The linear SVM has the lowest FPR (1.3%) while detecting almost no attacks.

Latency favors the neural models: CTI-GNN needs 0.0051 ms per sample, roughly 26 times less than RF (0.1329 ms). The run-to-run spread is large (0.109 ms for RF), so only the order of magnitude is meaningful.

C. Per-class behavior

Per-class results for the seed-42 run are computed on a 10,000-record evaluation sample drawn from the test split. Two patterns stand out. First, the neural models do not detect every class. CTI-GNN, the LSTM, and the SVM record zero recall on eventual-stop and random-offset attacks, and the ablation reaches a recall of only 0.015 on eventual stop and zero on random offset. RF, XGBoost, and KNN are the only models with non-zero recall on all five attack classes. Second, CTI-GNN is not uniformly worse. It reaches the highest F1 on random-position attacks (0.651 against 0.588 for RF) and on constant-position attacks (0.392 against 0.347), where it also outperforms the ablation (0.621 and 0.325). Its weakness is concentrated on the offset and stop classes, which are also the hardest classes for every model.

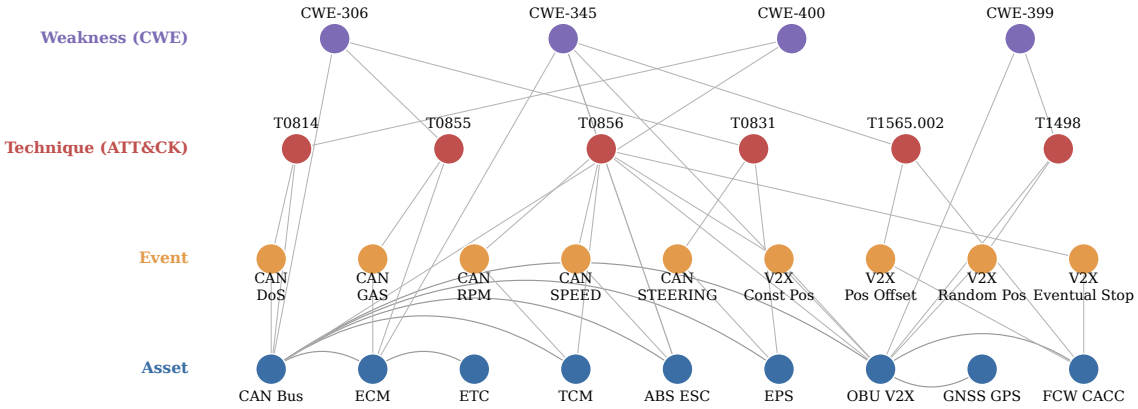


Fig. 1. Vehicle-threat graph exported from the pipeline (28 nodes, 46 edges). Nodes are grouped by type; lines show the typed relations listed in the text.

TABLE I
ATTACK CLASSES MAPPED TO ATT&CK TECHNIQUES AND CWE WEAKNESSES. IMPACT WEIGHTS ARE AUTHOR-ASSIGNED HEURISTICS.

Domain	Attack class	ATT&CK technique	CWE weakness	Impact
CAN	DoS	T0814 Denial of Service	CWE-400 Uncontrolled Resource Consumption	9.5
CAN	Gas spoofing	T0855 Unauthorized Command Message	CWE-306 Missing Authentication for Critical Function	9.2
CAN	RPM spoofing	T0856 Spoof Reporting Message	CWE-345 Insufficient Verification of Data Authenticity	8.0
CAN	Speed spoofing	T0856 Spoof Reporting Message	CWE-345 Insufficient Verification of Data Authenticity	8.5
CAN	Steering spoofing	T0831 Manipulation of Control	CWE-306 Missing Authentication for Critical Function	9.8
V2X	Const. position	T0856 Spoof Reporting Message	CWE-345 Insufficient Verification of Data Authenticity	8.1
V2X	Const. offset	T1565.002 Transmitted Data Manipulation	CWE-345 Insufficient Verification of Data Authenticity	8.3
V2X	Random position	T1498 Network Denial of Service	CWE-399 Resource Management Errors	7.8
V2X	Random offset	T1565.002 Transmitted Data Manipulation	CWE-345 Insufficient Verification of Data Authenticity	8.2
V2X	Eventual stop	T0856 Spoof Reporting Message	CWE-345 Insufficient Verification of Data Authenticity	9.0

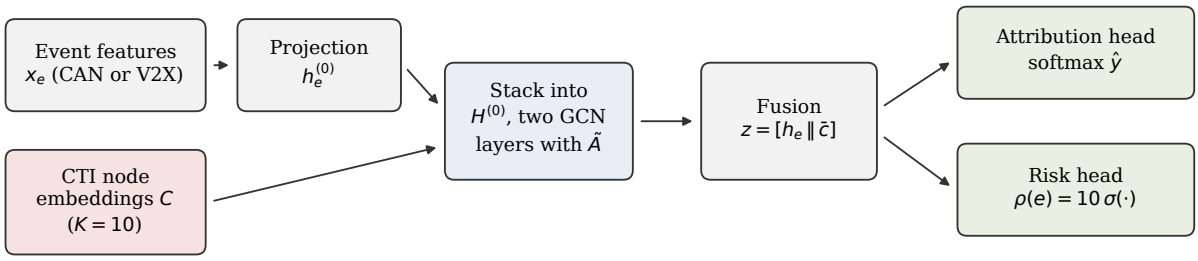


Fig. 2. CTI-GNN architecture. An event embedding is stacked with K learnable CTI node embeddings, passed through two graph convolutions, fused with the pooled CTI context, and read out by an attribution head and a risk head.

D. Is the graph worth its parameters?

The mean macro-F1 of CTI-GNN exceeds that of the ablation by 0.0088, against a seed-to-seed standard deviation of about 0.05 for CTI-GNN. Across 20 paired seeds this difference is not statistically significant, under either a paired t-test ($t = 0.91$, 19 degrees of freedom, $p = 0.372$) or a Wilcoxon signed-rank test [20], [21] ($W = 81$, $p = 0.388$). The same holds for the false positive rate (paired t-test $p = 0.708$, Wilcoxon $p = 0.498$) and for attribution accuracy (0.1607 against 0.1494; paired t-test $p = 0.422$, Wilcoxon $p = 0.546$). We therefore find no evidence that the graph improves detection, false alarms, or attribution accuracy over a network of the same kind without it.

A single-split analysis gives a different picture. On the seed-42 evaluation sample, McNemar's test [22] counts 168 records that CTI-GNN classifies correctly and the ablation does not, against 61 in the opposite direction ($\chi^2 = 49.07$, $p < 0.001$). A paired bootstrap [23] gives a macro-F1 difference of 0.042 with a 95% interval of [0.033, 0.052]. These tests treat test records as independent draws and ignore variation from training and initialization. Seed 42 is also a comparatively favorable seed for CTI-GNN (macro-F1 0.349 against its 20-seed mean of 0.306). The across-seed result, which includes that variation, is the more conservative one, and we treat the

TABLE II
VEREMI RESULTS (MEAN ± STD OVER 20 SEEDS). BEST VALUE IN EACH COLUMN IS IN BOLD. LATENCY IS BATCHED CPU INFERENCE TIME PER SAMPLE.

Model	Accuracy	Attribution acc.	FPR	Macro-F1 [95% CI]	Latency (ms)
Random forest	**0.7927 ± 0.0042**	0.2875 ± 0.0160	0.0531 ± 0.0064	**0.4595 ± 0.0127** [0.4536, 0.4654]	0.1329
XGBoost	0.7823 ± 0.0067	**0.2936 ± 0.0184**	0.0685 ± 0.0081	0.4560 ± 0.0163 [0.4484, 0.4636]	0.2429
KNN	0.7683 ± 0.0036	0.2142 ± 0.0092	0.0625 ± 0.0040	0.3819 ± 0.0082 [0.3781, 0.3857]	0.0483
CTI-GNN (proposed)	0.7794 ± 0.0066	0.1607 ± 0.0639	0.0318 ± 0.0143	0.3056 ± 0.0486 [0.2829, 0.3283]	0.0051
MLP (no graph)	0.7757 ± 0.0059	0.1494 ± 0.0292	0.0333 ± 0.0110	0.2968 ± 0.0221 [0.2865, 0.3071]	0.0007
LSTM	0.7697 ± 0.0057	0.1015 ± 0.0632	0.0264 ± 0.0170	0.2397 ± 0.0548 [0.2141, 0.2653]	0.0131
SVM (linear)	0.7694 ± 0.0043	0.0571 ± 0.0050	**0.0132 ± 0.0020**	0.2238 ± 0.0045 [0.2217, 0.2259]	0.0524

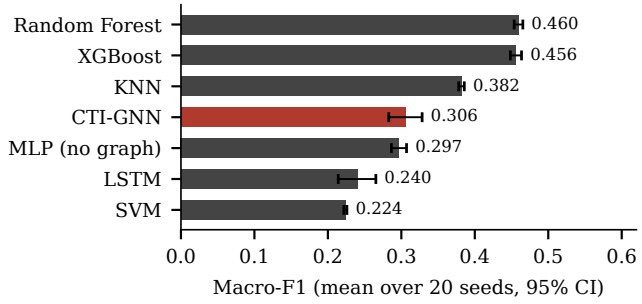


Fig. 3. VeReMi macro-F1 by model. Bars show the mean over 20 seeds and whiskers show the 95% confidence interval.

graph's accuracy benefit as unconfirmed.

E. Leakage audit on the CAN benchmark

The perfect CAN scores prompted three checks, summarized in Table III. First, 1,447 of the 8,999 test frames (16.08%) were exact duplicates of training frames. Removing them left 7,552 test frames, and all four models evaluated (KNN, RF, SVM, XGBoost) still scored 1.0. Second, dropping the three order-dependent features and keeping only the 14 per-frame features also left all four models at 1.0. These checks show that the high scores do not come from duplicates or frame ordering. They reflect the fact that the attack classes differ in arbitration ID and payload statistics, which makes the classes separable frame by frame.

The third check used a contiguous time-block split of the full 1.4 million frames instead of a stratified sample (985,750 training and 211,235 test frames, with models fit and scored on subsamples of 100,000 and 30,000 frames). Performance dropped sharply for most models: macro-F1 was 0.513 for RF, 0.603 for KNN, 0.605 for the SVM, and 0.803 for XGBoost, with test accuracy between 0.943 and 0.982. We attribute the drop to drift in the arbitration IDs used during injection in the RPM capture, which means that a model trained on early frames sees unfamiliar identifiers later in the file. A leave-one-attack-out experiment, in which an RF anomaly detector was trained without one attack class and asked to separate that class from benign traffic, scored 1.0 for all five held-out classes. Given the separability just described, we do not read this as evidence of zero-day generalization.

TABLE III
CAN LEAKAGE AND ORDERING CHECKS (MACRO-F1)

Protocol	KNN	RF	SVM	XGB
Stratified split (main)	1.000	1.000	1.000	1.000
Duplicates removed	1.000	1.000	1.000	1.000
Per-frame features only	1.000	1.000	1.000	1.000
Contiguous time block	0.603	0.513	0.605	0.803

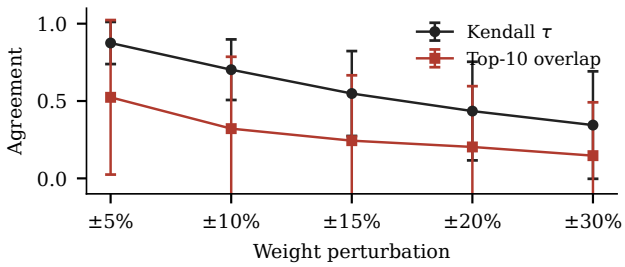


Fig. 4. Agreement between perturbed and baseline alert rankings over 1,000 trials. Markers show means and whiskers show one standard deviation.

F. Sensitivity of the alert ranking

Because the ranking follows from hand-assigned weights, we measured how stable it is. In 1,000 Monte Carlo trials, the criticality and impact weights were each perturbed by a relative amount of 5% to 30%, and the resulting ranking was compared with the unperturbed one (Fig. 4). Kendall's τ [24] fell from 0.875 at 5% to 0.703 at 10%, 0.435 at 20%, and 0.345 at 30%. The overlap between the top-10 alerts fell from 0.524 to 0.147 over the same range, with large standard deviations throughout. Small changes in the weights can therefore reorder the alerts an analyst would see first. Such weights should be elicited from domain experts before operational use.

VII. DISCUSSION

The results do not support the claim that adding CTI structure improves detection. Random forest and XGBoost perform best on VeReMi, and this is plausible given the features. Several of them are physical quantities with natural thresholds, such as the gap between claimed and implied speed, and trees can split on these directly, which is consistent with the tabular-data benchmarks of Grinsztajn et al. [9]. The neural models, trained on a split in which attacks make up

under a quarter of the records, tend to favor the benign class and the position-based attack types.

The architecture may also limit what the graph can add. The CTI embeddings are learned parameters that are shared by all events, so event-specific information reaches them only through message passing from the single event node. In that setting the graph behaves like a learned, fixed context vector, which may explain why its effect is small. We have not tested this explanation directly.

What the graph does provide is a structured explanation. Each predicted class resolves to a technique, a weakness, and a set of affected assets, which is the information an operator needs for triage and which a bare classifier cannot give. That benefit is real but depends on the quality of the mapping, which here is author-assigned.

A. Limitations

The study has several limits. Only density 3 of VeReMi was evaluated, and VeReMi is simulated, so transfer to real V2X traffic is untested. The CAN experiments use a balanced subsample of the full capture, and the time-block result shows that conclusions from that sample do not carry over to a more realistic split. The CTI mapping, the impact weights, and the asset weights are heuristic and unreviewed, and the mapping mixes ICS and Enterprise techniques. The risk head is not validated against labeled severity. The per-class analysis uses a single seed and a 10,000-record sample, and no experiments were run on a real vehicle. The search budget of four configurations per model is small and may disadvantage the neural models more than the tree ensembles.

VIII. Conclusion

We built a vehicle-threat knowledge graph, trained a graph-convolutional classifier on top of it, and evaluated the result against five baselines and an ablation on in-vehicle and V2X benchmarks under a protocol designed to avoid optimistic estimates. Tree ensembles outperformed the proposed model on V2X data by about 0.15 macro-F1, the advantage of the graph over an ablation was not significant across 20 seeds, and the CAN benchmark proved saturated under the standard sampling protocol and much harder under a time-block split. The graph does give each alert a technique, a weakness, and a set of affected assets, and we regard that as its main contribution. Future work should validate the mapping and weights with domain experts, extend the evaluation to higher VeReMi densities and real vehicle data, address class imbalance in the neural models, and test designs in which CTI nodes interact with event-level evidence.

References

[1] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesner, and T. Kohno, "Comprehensive experimental analyses of automotive attack surfaces," in *Proc. 20th USENIX Security Symp.*, San Francisco, CA, USA, 2011.

[2] W. Wu, R. Li, G. Xie, J. An, Y. Bai, J. Zhou, and K. Li, "A survey of intrusion detection for in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 919–933, Mar. 2020.

[3] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, McLean, VA, USA, Tech. Rep., 2020, revised March 2020.

[4] MITRE Corporation, "Common weakness enumeration (CWE)," https://cwe.mitre.org/, accessed: Nov. 2026.

[5] E. C. P. Neto, H. Taslimasa, S. Dadkhah, S. Iqbal, P. Xiong, T. Rahman, and A. A. Ghorbani, "CICIoV2024: Advancing realistic IDS approaches against DoS and spoofing attack in IoV CAN bus," *Internet of Things*, vol. 26, p. 101209, 2024.

[6] R. W. van der Heijden, T. Lukaseder, and F. Kargl, "VeReMi: A dataset for comparable evaluation of misbehavior detection in VANETs," in *Security and Privacy in Communication Networks (SecureComm 2018)*, ser. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 254. Cham: Springer, 2018, pp. 318–337.

[7] S. So, P. Sharma, and J. Petit, "Integrating plausibility checks and machine learning for misbehavior detection in VANET," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2018, pp. 564–571.

[8] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022.

[9] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, vol. 35, 2022.

[10] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[11] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[12] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[15] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[17] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python Sci. Conf. (SciPy)*, Pasadena, CA, USA, 2008, pp. 11–15.

[18] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.

[19] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations (ICLR)*, Toulon, France, 2017.

[20] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[21] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[22] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[23] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1993.

[24] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.